Automation for the course “Programming with Python”

Lyuboslav Karev & Alexander Ignatov

Who are we ?

In case you don’t already know

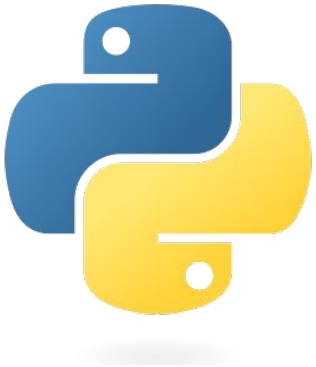


Lyubo



Alex





Програмиране с Python

2022 - 20XX

Pytoni

The course chatbot.



/ask question: Какво имаме днес ?

Любо used ask

Pytoni APP 15.10.26, 9:27

Днес (четвъртък, **15.10.2026**) няма редовна лекция — вместо това ще се проведе **workshop (упражнение)** по теми **1 и 2** (въведение в Python и основи: променливи, типове, разклонения, цикли, функции). Започва в 19:00, зала 101.

Полезни ресурси:

- Календар на курса
- Материали на курса

Успех на работилницата! 🍫

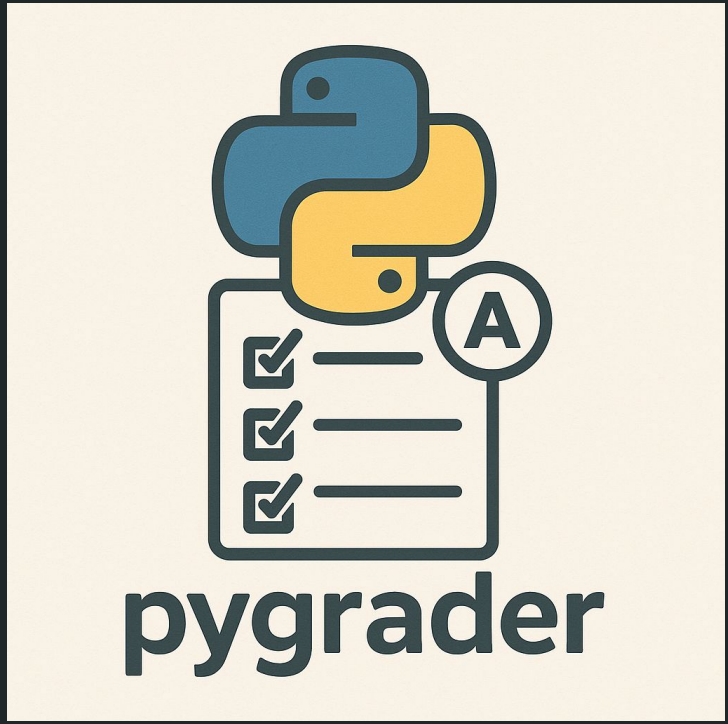
Google Docs

Calendar 2026



Pygrader

An overengineered grader for Python projects



What is the pygrader ?

- Each project was graded on pylint, mypy and unittest coverage scores.
- Running required a virtual environment to be created as well.
- If we could easily switch which checks are run, that would have been great.

Types of checks



```python
class ScoredCheck(AbstractCheck[float]):
    """Each scored check has a maximum amount of points."""

    def __init__(
        self,
        name: str,
        max_points: int,
        project_root: str,
        is_venv_requred: bool = False,
        env_vars: dict[str, str] | None = None,
        assets: list[str] | None = None,
    ):
        ...

    @property
    def max_points(self) -> int:
        ...
```

```python
class AbstractCheck[T](ABC):
    """Each check has a name and a project root path."""

    def __init__(
        self,
        name: str,
        project_root: str,
        is_venv_required: bool = False,
        env_vars: dict[str, str] | None = None,
        assets: list[str] | None = None,
    ):
        ...

    @abstractmethod
    def run(self) -> CheckResult[T]:
        ...

    @property
    def name(self) -> str:
        ...

    @property
    def env_vars(self) -> dict[str, str] | None:
        ...

    @property
    def assets(self) -> list[Resource]:
        ...

    @staticmethod
    def is_running_within_venv() -> bool:
        ...

    def _pre_run(self) -> None:
        ...
```

```python
class NonScoredCheck(AbstractCheck[bool]):
    """Non-scored checks do not have a maximum amount of points."""

    def __init__(
        self,
        name: str,
        project_root: str,
        is_fatal: bool,
        is_venv_requred: bool = False,
        env_vars: dict[str, str] | None = None,
        assets: list[str] | None = None,
    ):
        ...

    @property
    def is_fatal(self) -> bool:
        ...
```

Configurations

- Regular JSON files.
- We can define what checks are ran, along with their arguments.
- Templates are supported in the configuration files.



```
{
    "environment": {
        "variables": {
            "GLOBAL_VAR": "global_value",
            "API_KEY": "global_api_key"
        }
    },
    "checks": [
        {
            "name": "structure",
            "is_venv_required": false,
            "structure_file": "${{config_dir}}/project_structure.json"
        },
        {
            "name": "pylint",
            "max_points": 3,
            "is_venv_required": true,
            "pylintrc_path": "${{config_dir}}/2024.pylintrc"
        },
        {
            "name": "type-hints",
            "max_points": 3,
            "is_venv_required": false
        },
        {
            "name": "tests",
            "max_points": 13.5,
            "is_venv_required": true,
            "tests_path": [
                "https://raw.githubusercontent.com/example/repo/main/tests/test_sample.py"
            ],
            "default_test_score": 1,
            "test_score_mapping": {
                "TestExample": 0.5,
                "test_function": 2
            },
            "environment": {
                "variables": {
                    "TEST_VAR": "test_value"
                }
            }
        }
    ]
}
```

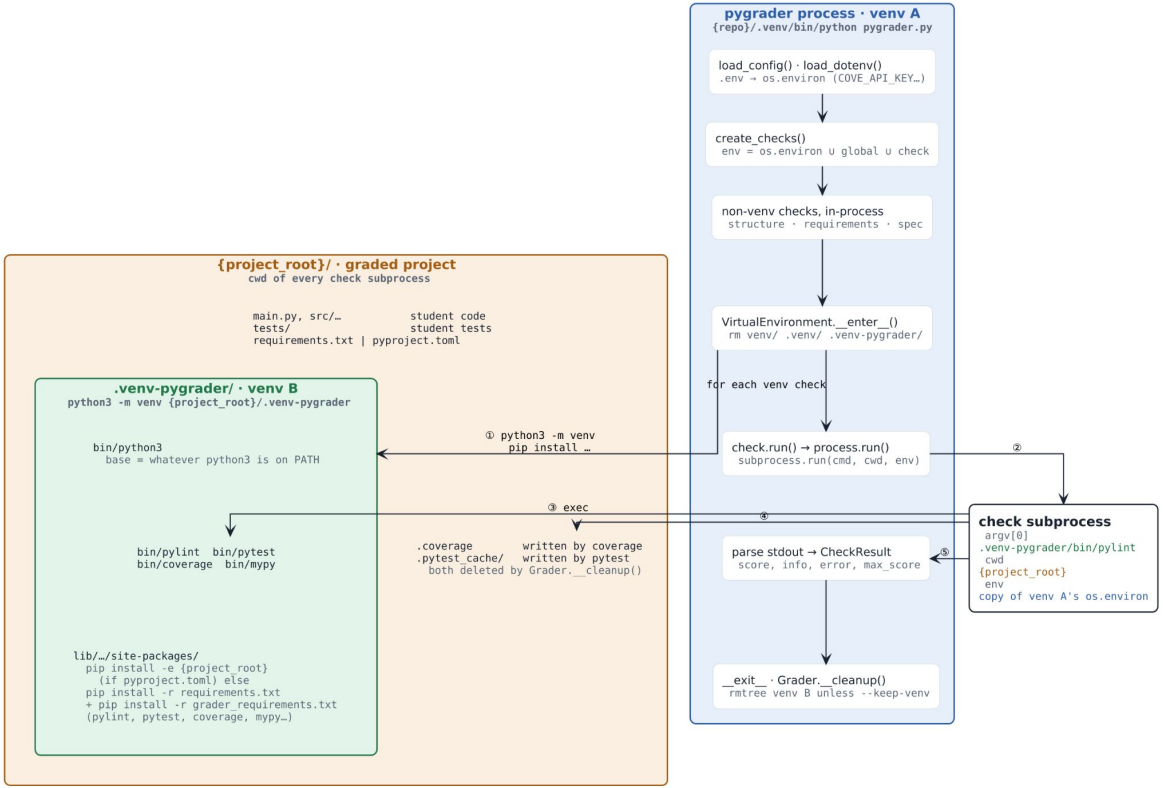
Loading resources

- Tests, structure descriptions, specifications
- Can be loaded locally, from a URL or from a specific object store.

Running the checks

- Most checks are running as a subprocess.
- From inside the virtual environment of the project.
- Environment variables are modified.

Running the checks



Results

```python
@dataclass
class CheckResult[T]:
    """Class representing the result of a check."""

    name: str
    result: T
    info: str
    error: str


@dataclass
class ScoredCheckResult(CheckResult[float]):
    """Class representing the result of a scored check."""

    max_score: int


@dataclass
class NonScoredCheckResult(CheckResult[bool]):
    """Class representing the result of a non-scored check."""
```

```python
@dataclass
class GradingResult:
    run_id: str
    total_score: float
    max_score: float
    results: list[CheckResult]
```

```python
class ResultsReporter(ABC):
    def __init__(self, is_verbose: bool = False) -> None:
        ...

    @abstractmethod
    def to_string(self) -> str:
        ...

    def add_result(self, result: GradingResult) -> None:
        ...

class JSONResultsReporter(ResultsReporter):
    def to_string(self) -> str:
        ...

class CSVResultsReporter(ResultsReporter):
    def to_string(self) -> str:
        ...

class PlainTextResultsReporter(ResultsReporter):
    def to_string(self) -> str:
        ...
```

Available checks

- Coverage - measure test code coverage
- Requirements - check if the project has a requirements.txt/pyproject.toml and is installable
- Type hints - check for the % of code covered by type hints
- Structure - check if the project follows a specific directory structure
- Tests - Execute a set of tests
- Specification - Check if the project has implemented a set of functionalities

Getting to students

- pygrader-web
- Docker image
- Directly from source

Short demo

Homework-deployer

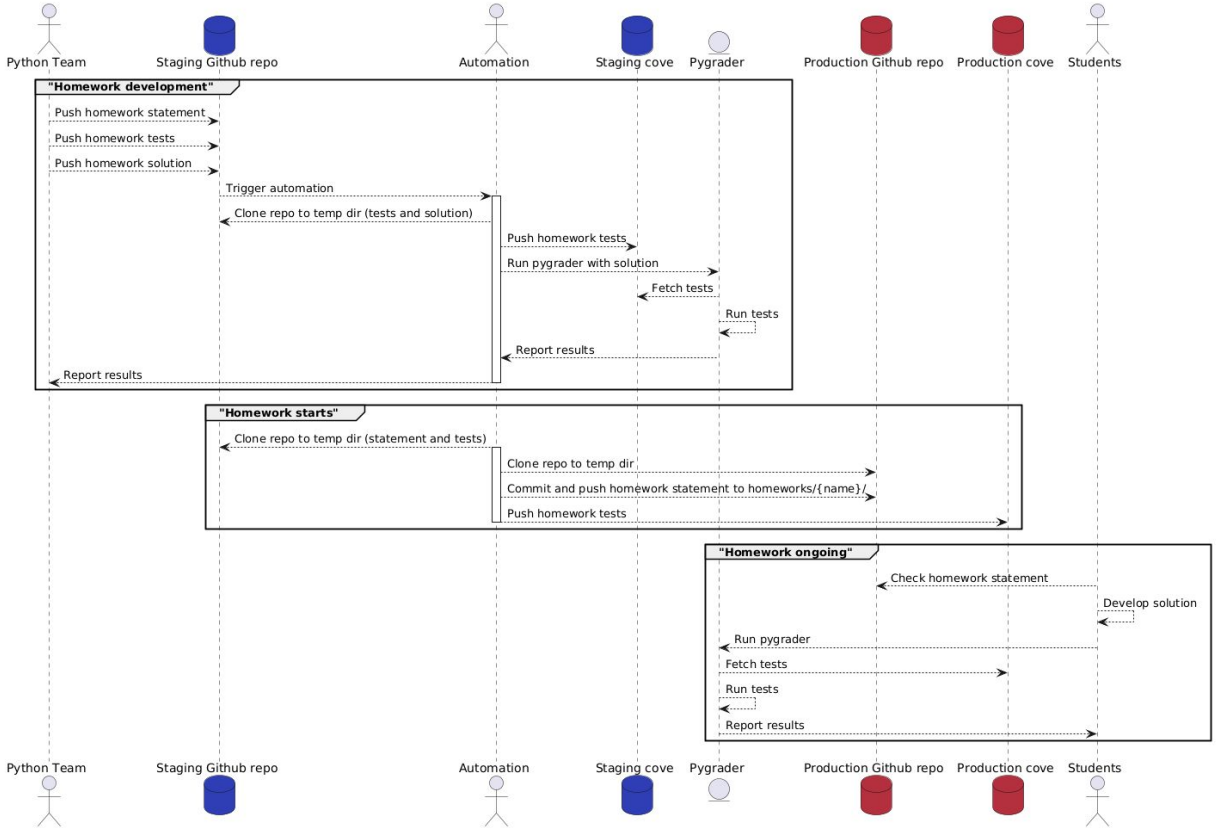
Uploading homeworks at 1am,
while we are sleeping



What is homework-deployer ?

- Homeworks are given out as Markdown files, plus a pygrader configuration file.
- We write different tasks each year.
- During “development” we want to test if the pygrader config works on a valid solution.
- Releasing homeworks could happen automatically.

How it works ?


Python Team
Staging Github repo
Automation
Staging cove
Pygrader
Production Github repo
Production cove
Students
"Homework development"
Push homework statement
Push homework tests
Push homework solution
Trigger automation
Clone repo to temp dir (tests and solution)
Push homework tests
Run pygrader with solution
Fetch tests
Run tests
Report results
Report results
"Homework starts"
Clone repo to temp dir (statement and tests)
Clone repo to temp dir
Commit and push homework statement to homeworks/{name}/
Push homework tests
"Homework ongoing"
Check homework statement
Develop solution
Run pygrader
Fetch tests
Run tests
Report results
Python Team
Staging Github repo
Automation
Staging cove
Pygrader
Production Github repo
Production cove
Students

Deployment

Where is everything running ?



How do you package it and where do you run it ?

- We all know we can package Python as wheel, a binary, or just run it in Docker.
- We’ve picked the Docker approach.
- It’s easy to set up and migrate in case the server starts burning

Speed round ?

- Lecture materials are Jupyter Notebooks - automatically built into a Jupyter Book
- Students grades are a spreadsheet - REST API to interact with it
- Topic selection for the final projects - simple frontend + PocketBase as a backend



pygrader





...and all others things



Links:

- https://github.com/fmipython
- https://github.com/fmipython/pygrader
- https://github.com/fmipython/pytoni
- https://github.com/fmipython/homework-deployer
- https://www.linkedin.com/in/lyuboslav-karev-33357715a/
- https://www.linkedin.com/in/alexander-ignatov-yalishanda/